

Reviewer Pack — Mostar Index of Gate-Adjacency Graph Bounds $ACC^0[m]$ Size fo...

Entry b7ec55b48398 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 08:56:29 UTC

Mostar Index of Gate-Adjacency Graph Bounds $ACC^0[m]$ Size for MOD_q

Entry ID: b7ec55b48398 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 08:56:29 UTC

1. Conjecture

Field A (mathematical branch): Extremal chemical graph theory — the Mostar index $Mo(G) = \sum_{uv \in E(G)} |n_u(uv) - n_v(uv)|$ where $n_u(uv)$ counts vertices strictly closer to u than to v (Došlić-Martinjak-Škrekovski-Spuženić-Tepeh 2018 ‘Mostar index’; Tratnik 2019; Arockiaraj-Klavžar 2020; Hayat-Liu 2021). $Mo(G)$ is a structural distance-balance invariant of finite graphs, computed from all-pairs shortest paths and absolute differences — non-ring operations that do NOT extend to polynomial relaxations over F_q , shielding the invariant from Aaronson-Wigderson algebrization. The bridge to circuit complexity has essentially zero presence in the literature: an arXiv search for ‘Mostar index’ AND (‘ ACC^0 ’ OR ‘circuit complexity’ OR ‘Williams algorithmic method’ OR ‘Boolean circuit’) returns 0 direct hits and <5 adjacent papers, all on chemical isomer enumeration. Distinct from blacklisted Möbius/independence/Birkhoff/Plünnecke attempts: Mo measures edge-wise distance asymmetry on the circuit DAG’s underlying graph, not poset incidence values, simplicial Euler characteristics, doubly-stochastic Sinkhorn limits, or top-Fourier doubling. **Field B** (complexity object): $ACC^0[m]$ circuit size for MOD_q on n inputs with q coprime to m (focus $q=3, m=2$), in the Williams 2011 / Murray-Williams 2018 algorithmic-method regime targeting $NEXP \not\subseteq ACC^0$ and explicit-function ACC^0 lower bounds. The invariant is a STRUCTURAL property of the circuit DAG (two $ACC^0[m]$ circuits computing the same function can have very different Mo values), shielding from Razborov-Rudich natural-proofs.

Statement:

For an $ACC^0[m]$ circuit C with s gates and depth d , let G_C be the underlying undirected adjacency multigraph on gate vertices (with each wire contributing one edge) and let $Mo(C) := \sum_{uv \in E(G_C)} |n_u(uv) - n_v(uv)|$ denote its Mostar index, with $n_u(uv) = |\{w : d_{G_C}(w, u) < d_{G_C}(w, v)\}|$. Conjecture: (i) for every $ACC^0[m]$ circuit C of size s and depth d , $Mo(C) \leq C_1 \cdot s^2 \cdot d$ for an absolute constant $C_1 \leq 4$; (ii) for every $ACC^0[2]$ circuit C computing MOD_3 on n inputs ($n \geq 5$), $Mo(C) \geq C_2 \cdot n \cdot s^{\{1/2\}}$ for $C_2 \geq 1/16$. A single $ACC^0[2]$ circuit violating (i), or a MOD_3 $ACC^0[2]$ circuit violating (ii), falsifies the conjecture.

Rationale (proposer’s reasoning):

Williams-style structural exploitations succeed by extracting circuit-wiring properties that are necessarily small for fast circuit-SAT yet large on hard symmetric functions; $Mo(C)$ measures cumulative distance-balance asymmetry, which is forced to be tame on shallow, locally-concentrated ACC wirings (clause i) but forced to spread on circuits that must

reach every input to compute a fully-symmetric function like MOD₃ (clause ii). The invariant uses no ring operations and is not a truth-table functional, so it evades both algebrization and natural-proofs. The distance-balance angle is genuinely new to circuit complexity — Mostar’s recent introduction (2018) has kept it confined to chemical graph theory.

Taxonomy category: ACC_{LB} via WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a71553afb6601c64

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 360 random ACC⁰[2] circuits (30 seeds × 3 depths × 4 sizes) and ≥20 verified MOD₃ ACC⁰[2] circuits (enumerated $s \leq 12, n \leq 8$ plus 30 padded size-40), compute Mo(C) via BFS. SUPPORTED iff every random instance satisfies $\text{Mo}(C) \leq 4 \cdot s^2 \cdot d$ AND ≥80% of MOD₃ instances satisfy $\text{Mo}(C) \geq (1/16) \cdot n \cdot s^{\{1/2\}}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.86	SAFE	SAFE
ALGEBRIZATION	SAFE	0.84	SAFE	SAFE
KARP_LIPTON	SAFE	0.85	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -Mostar index circuit complexity lower bound -distance-balanced graph ACC⁰ MOD circuit -Mostar index Boolean circuit DAG gate adjacency

Top relevant hits considered: - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/1606.05050v1>] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [<http://arxiv.org/abs/2504.19966v3>] Quantum circuit lower bounds in the magic hierarchy - [<http://arxiv.org/abs/0711.2360v2>] Every longest circuit of a 3-connected, $K_{3,3}$ -minor free graph has a chord - [<http://arxiv.org/abs/1012.3942v1>] Distance-balanced closure of some graphs - [<http://arxiv.org/abs/1102.0127v1>] On the Connectivity of Bipartite Distance-Balanced Graphs - [<http://arxiv.org/abs/2005.09416v1>] Mostar index of graph operations - [<http://arxiv.org/abs/2407.04826v1>] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
from collections import deque

def matrix_mult(a, b):
    return [[sum(a[i][k] * b[k][j] for k in range(len(b))) for j in range(len(b[0]))] for i in range(len(a))]

def matrix_pow(mat, power):
    result = [[1 if i == j else 0 for j in range(len(mat))] for i in range(len(mat))]
    while power > 0:
        if power % 2 == 1:
            result = matrix_mult(result, mat)
            mat = matrix_mult(mat, mat)
            power //= 2
    return result

def bfs(graph, start):
    distances = {node: -1 for node in graph}
    distances[start] = 0
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbor in graph[current]:
            if distances[neighbor] == -1:
                distances[neighbor] = distances[current] + 1
                queue.append(neighbor)
    return distances

def generate_acc0_circuit(seed, s, d, n=0, is_mod3=False):
    random.seed(seed)
    gates = []
    adj = {i: set() for i in range(s)}
    gate_types = ['AND', 'OR', 'MOD_2'] if not is_mod3 else ['AND', 'OR', 'MOD_2', 'MOD_3']

    for i in range(s):
        gate_type = random.choice(gate_types)
        fan_in = random.randint(2, min(3, math.ceil(math.log2(s))))
        inputs = []
        for _ in range(fan_in):
            if i == 0:
                inputs.append(random.randint(0, n-1))
            else:
                inputs.append(random.randint(0, i-1))
        gates.append((gate_type, inputs))
        for inp in inputs:
            adj[i].add(inp)
            adj[inp].add(i)

    return gates, adj

def compute_mostar_index(gates, adj):
    s = len(gates)
    mo = 0
    for u in range(s):
```

```

distances_u = bfs(adj, u)
for v in adj[u]:
    if v > u:
        distances_v = bfs(adj, v)
        n_u = sum(1 for w in range(s) if distances_u[w] < distances_v[w])
        n_v = sum(1 for w in range(s) if distances_v[w] < distances_u[w])
        mo += abs(n_u - n_v)
return mo

def run_trial(seed):
    random.seed(seed)
    trials = []
    sizes = [10, 20, 30, 40]
    depths = [2, 3, 4]

    for s in sizes:
        for d in depths:
            gates, adj = generate_acc0_circuit(seed, s, d)
            mo = compute_mostar_index(gates, adj)
            bound = 4 * s * s * d
            conjecture_holds = mo <= bound
            counterexample = f"Random ACC^0[2] circuit with s={s}, d={d} violated bound: Mo(C)={mo}"

            trials.append({
                "seed": seed,
                "metric_name": "Mostar Index",
                "metric_value": mo,
                "instances_tested": 1,
                "conjecture_holds": conjecture_holds,
                "counterexample": counterexample,
                "s": s,
                "d": d
            })

# Add MOD_3 circuits
mod3_circuits = []
for s in [10, 20, 30, 40]:
    for n in [5, 6, 7, 8]:
        gates, adj = generate_acc0_circuit(seed, s, 2, n, is_mod3=True)
        mo = compute_mostar_index(gates, adj)
        bound = (1/16) * n * math.sqrt(s)
        conjecture_holds = mo >= bound
        counterexample = f"MOD_3 ACC^0[2] circuit with s={s}, n={n} violated bound: Mo(C)={mo}"

        mod3_circuits.append({
            "seed": seed,
            "metric_name": "Mostar Index",
            "metric_value": mo,
            "instances_tested": 1,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample,
            "s": s,
            "n": n
        })

# Combine trials

```

```

all_trials = trials + mod3_circuits

# Calculate statistics
metric_values = [trial["metric_value"] for trial in all_trials]
mean = sum(metric_values) / len(metric_values)
std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
support_fraction = sum(1 for trial in all_trials if trial["conjecture_holds"]) / len(all_trials)

# Check for counterexamples
counterexamples = [trial for trial in all_trials if not trial["conjecture_holds"]]
if counterexamples:
    first_failing_seed = counterexamples[0]["seed"]
    return {
        "metric_name": "Mostar Index",
        "metric_value": mean,
        "instances_tested": len(all_trials),
        "conjecture_holds": False,
        "counterexample": counterexamples[0]["counterexample"],
        "first_failing_seed": first_failing_seed
    }

return {
    "metric_name": "Mostar Index",
    "metric_value": mean,
    "instances_tested": len(all_trials),
    "conjecture_holds": True,
    "counterexample": "",
    "mean": mean,
    "std": std,
    "support_fraction": support_fraction
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17]
    trials = []

    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        trials.append(trial)

# Calculate overall statistics
metric_values = [trial["metric_value"] for trial in trials]
mean = sum(metric_values) / len(metric_values)
std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
support_fraction = sum(1 for trial in trials if trial["conjecture_holds"]) / len(trials)

# Check for counterexamples
counterexamples = [trial for trial in trials if not trial["conjecture_holds"]]
if counterexamples:
    first_failing_seed = counterexamples[0]["seed"]
    print(f'RESULT: FALSIFIED counterexample="{counterexamples[0]["counterexample"]}" first_failing_seed={first_failing_seed}')
elif support_fraction >= 0.8:
    print(f'RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}')
else:
    print('RESULT: INCONCLUSIVE reason=insufficient_support')

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fed4589a.py", line 160, in <module>
    trial = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fed4589a.py", line 84, in run_trial
    gates, adj = generate_acc0_circuit(seed, s, d)
                  ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fed4589a.py", line 53, in generate_acc0
    inputs.append(random.randint(0, n-1))
                   ~~~~~
```

```
File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ~~~~~
```

```
File "/usr/lib/python3.12/random.py", line 319, in randrange
    raise ValueError(f"empty range in randrange({start}, {stop})")
ValueError: empty range in randrange(0, 0)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ValueError in generate_acc0_circuit (empty range in randrange(0, 0)) before producing any data, so neither support nor falsification can be assessed. | next: Fix the circuit generator to guard against n=0 (ensure input count ≥ 1 before sampling) and rerun the 360 random + ≥ 20 MOD_3 instance battery.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	200158
2	preregistration	claude_max	opus	0	0	6499
3	novelty	claude_max	opus	0	0	4025
4	novelty	claude_max	opus	0	0	8073
5	test_gen	mistral	codestral-latest	0	0	370297
6	test_gen	mistral	codestral-latest	0	0	317956
7	test_gen	mistral	codestral-latest	0	0	315834
8	test_gen	mistral	codestral-latest	0	0	319050

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	claude_max	opus	0	0	4369

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1546261 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in research/audit/b7ec55b48398.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b7ec55b48398.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b7ec55b48398.tar.gz (if generated)